

Development of a Deep Learning Based Automatic License Plate Recognition System

^aEdifon Emmanuel J., ^bOsa Edosa, ^cOrukpe Patience E.

^{a, b, c} Department of Electrical and Electronic Engineering, Faculty of Engineering, University of Benin,
P.M.B. 1154, Benin City, Nigeria.

*Corresponding author: edosa.osa@uniben.edu

ABSTRACT

Due to the many benefits of vehicle license plate detection and recognition, ranging from security to logging of personnel entry, automatic systems which ensure these functions are in high demand. Artificial intelligence and in particular Deep Learning techniques have proven to be highly efficient for object detection tasks. Hence, implementation of such methods for detecting the characters in license plates is an active area of research within the computer vision ecosystem. This paper describes the development of a Deep Learning based Automatic License Plate Recognition system. Two deep learning models were employed in developing the object detection phase, namely MobileNetV2 and YOLOv9c. The Automatic Number Plate Recognition dataset was used to train the MobileNetV2 model, while the Automatic Number Plate Recognition dataset and the Car License Plate Detection dataset were used together for training the YOLOv9c model. Testing of the deep learning pipeline was via images from Google and the Automatic Number Plate Recognition dataset. The system performed satisfactorily, with the YOLOv9c component being able to detect license plate existing deep in the background of images.

Keywords: *Computer vision, Deep learning, YOLOV9c, Convolution Neural Network.*

INTRODUCTION

An Automated License Plate Recognition (ALPR) system can be referred to as a computer vision dependent technology which employs machine learning algorithms to detect license plates and optical character recognition (OCR) engines for automatic identification of vehicles. ALPR systems collect images and sometimes videos from multiple cameras to serve as data. Such data, is further processed and analyzed by cross reference against security systems, integrated databases, parking management systems, traffic management systems and other related sources. Automated License Plate Recognition (ALPR) systems usually function in three prongs namely license plate localization (or detection), character segmentation and character recognition [1]. In the localization stage, the number plate is detected and cropped from the given image frame for character recognition. In the next stage, the cropped license plates (LPs) are further processed by segmenting individual characters from the detected LP to serve as inputs for a character recognition engine [2]. Nonetheless, both character segmentation and recognition could sometimes be considered as a single task in some systems. According to Persistence Market Research (PMR), the global market for automatic number plate recognition systems, which was valued at 3.1 billion U.S dollars in 2022, is projected to attain a mark of 6.8 billion U.S. dollars by 2032. The wide plethora of benefits that smart license plate recognition systems offer to many users is the reason for the forecasted surge. An ALPR system could be integrated into traffic monitoring systems where they work together with traffic lights for identification of traffic offenders or flagged vehicles [3]. Other benefits include enhanced security, law enforcement, defense, border control, as well as improved customer experience and reduction in overall costs. Due to the benefits of deep learning in contemporary artificial intelligence (AI) applications, AI-powered license plate recognition systems have become commonplace since they are fast and highly

accurate. Deep neural networks have proven to be optimal choices for object detection tasks [2], hence such algorithms are being trained on datasets which consist of images derived from various multiple sources and with different qualities. This therefore helps to improve the accuracy, performance, scalability as well as cost-efficiency of license plate recognition systems. Many of such advanced LPR systems have the ability to recognize plates of fast-moving cars in real time and from multiple cameras. They also can operate under challenging conditions such as poor lighting and extreme weather [4].

The primary objective of this paper is the development of a deep learning based license plate detection and recognition system pipeline.

Below is a summary of the contributions made by this paper:

1. Implementation of OpenCV 2 for image processing on the Car Detection and Automatic Number Plate Recognition datasets.
2. Implementation of MobileNetv2 and You Only Look Once Version 9-compact (YOLOv9c) for license plate object detection.
3. Normalization of bounding box coordinates in the Automatic Number Plate Recognition dataset.
4. Determination of loss function as RMSE (Root Mean Square Error).
5. Implementation of Pytesseract for optical character recognition.
6. Development of automatic license plate recognition system user interface via FastAPI and SQLAlchemy.

2. LITERATURE REVIEW

2.1 Related Works

License plate detection, character segmentation and recognition tasks remain challenging and important research components of the computer vision ecosystem. Various researchers have thus contributed in this regard.

Authors in [5] presented a robust and efficient Automatic License Plate Recognition (ALPR) system based on the YOLO object detector. The resulting approach employed two datasets, the SSIG dataset, containing 2,000 frames from 101 vehicle videos and a novel UFPR-ALPR dataset containing 150 videos and 4,500 frames for different types of vehicles such as buses, cars, motorcycles, and trucks. They achieved a recognition rate of 93.53% and 47 Frames Per Second (FPS) with SSIG. With their proposed UFPR-ALPR dataset, trial versions of commercial systems achieved recognition rates below 70%, while their deep learning technique performed better, with a recognition rate of 78.33% and 35 FPS. This work was limited to the license plates in Brazil. In [2], a novel memory and time-efficient automatic license plate recognition (ALPR) system was developed using YOLOv5. YOLOv5, also dubbed v5small, consists of 283 layers, 16.4 GFLOPS, and approximately 7 million parameters. Firstly, a custom transfer learned model was utilized for detecting the license plates and secondly, an LSTM-based OCR engine (Easy-OCR) was used for recognition. They employed a dataset containing images from the Google open images dataset as well as the Indian License plate dataset. Along with YOLOv5, YOLOv4 models were also trained on the same dataset for performance comparison. The proposed ALPR system resulted in a mean average precision of 87.2% and 4.8 millisecond testing time on still images. The entire detection and recognition process took approximately 85 milliseconds. Author in [6] introduced the JALPR dataset for automatic license plate recognition in the country of Jordan. Two-stage Convolutional Neural Networks (CNNs) were employed with the YOLO 3 architecture. Experimental results via real videos showed that developed system could recognize Jordanian plates with high efficiency. It achieved 87 percent recognition accuracy, as against

commercial systems of OpenALPR and Sighthound with 81 percent and 65 percent recognition accuracies respectively. In [3] a license plate recognition system (without detection) was implemented for Thailand license plates. A two-step approach of training and recognizing via deep learning techniques of Inception-v3 and MobileNets was first used to segment and train the three lines in the Thailand license plates. In the second step, the Thai letters and numbers on the plate were segmented and trained. The models from these trainings were deployed with the whole license plate testing. A test was carried out on LP images with varied angles of orientation and the accuracy rate of line and character recognition were 96.94% and 91.76% respectively. In [7], Open CV and Python were implemented for license plate detection and character segmentation. Their license plate recognition system involved a website where visitors can submit registration information, followed by a display of the car information. The work was specific to Indian license plates. Authors in [8] developed an ALPR system with YOLOv5 for number plate detection and used Google Tesseract for Character Recognition. After training over 100 epochs, their proposed system yielded 0.943 Recall and 0.629 Precision results respectively. Authors in [9] presented a You Only Look Once (YOLOv3) based ALPR system. The system had excellent results during training, however when tested on images with multiple vehicles, it could not detect the license plates of vehicles at the background. Many of the works considered in literature propose YOLO as a popular deep learning model for license plate detection, hence in this work YOLOv9c (YOLOv9-compact) has been adopted along with MobileNetv2 deep learning model for license plate detection. The properties of YOLOv9c model are summarized as follows: 618 layers, 25530003 parameters, 25529987 gradients and 103.7 GFLOPs presenting higher resources and capacity than the earlier version of YOLO employed in considered literature above. An application has also been developed for users of the system. Hence, our solution can thus be deployed on a general purpose or specialized computer as well as mobile devices.

2.2 MobileNetV2

MobileNetV2 [10] is a fast mobile-based convolutional neural network architecture developed by Google, as an extension from MobileNets [11]. It enables real-time classification in computationally constrained mobile devices such as smartphones. Although trained from ImageNet, this model can be transferred as a pre-trained model to any relevant dataset. The MobileNetV2 architecture has two main components. The first being an inverted residual block, while the second is a linear bottleneck block. MobileNetV2 is a major advancement over MobileNetV1 in terms of classification, object identification, and semantic segmentation for mobile visual recognition. Nonetheless, the Depthwise Separable Convolutions characteristic of MobileNetV1 also features in MobileNetV2.

A. Depthwise Separable Convolutions

Depthwise Separable Convolutions are a key building block for many efficient neural network architectures [11]. The basic concept in this case, is to replace a full convolutional operator with a factorized version that splits the convolution into two separate layers. The first layer known as a *depthwise* convolution, performs lightweight filtering by applying a single convolutional filter per input channel. The second layer is a 1×1 convolution (pointwise convolution), which functions to build new features via computing linear combinations of available input channels. The computational cost for Depthwise separable convolutions is given by Equation 1 as:

$$h_i \cdot w_i \cdot d_i(k^2 + d_j) \quad (1)$$

Equation 1 represents the sum of the depthwise and 1×1 pointwise convolutions. Effectively, depthwise separable convolution reduces computation compared to traditional layers by almost a factor of $\frac{k^2 d_j}{(k^2 + d_j)}$.

MobileNetV2 uses $k = 3$ (3×3 depthwise separable convolutions) to make the cost of computation eight to nine times smaller than that of standard convolutions [9].

B. Linear Bottleneck

In general, if a result of a layer transformation $ReLU(Bx)$ has a non-zero volume S , the points mapped to interior S are obtained via a linear transformation B of the input, thus indicating that the part of the input space corresponding to the full dimensional output, is limited to a linear transformation. In other words, deep networks only have the power of a linear classifier on the non-zero volume part of the output domain. On the other hand, when ReLU collapses the channel, it inevitably loses information in that channel. However, if many channels are available, and a structure exists in the activation manifold, that information could still be maintained or preserved in the other channels. Furthermore, if the input manifold could be embedded into a significantly lower-dimensional subspace of the activation space, the ReLU transformation can preserve the information while introducing the required complexity into the set of expressible functions. Neural architectures can thus be optimized by inserting linear bottleneck layers into the convolutional blocks to prevent nonlinearities from destroying too much information.

C. Inverted residuals

The bottleneck blocks appear similar to residual blocks where each block contains an input followed by several bottlenecks then followed by expansion [12]. Shortcuts can be inserted directly between the bottlenecks based on the intuition that bottlenecks actually contain all the necessary information, while an expansion layer acts merely as an implementation detail that accompanies a non-linear transformation of the tensor. Figure 1 shows the difference between residual block and inverted residual [12]. Despite the similarity in intent for inserting shortcuts in both the classical and inverted residual connections, namely, improving the ability of a gradient to propagate across multiplier layers, the inverted design is considerably more memory efficient.

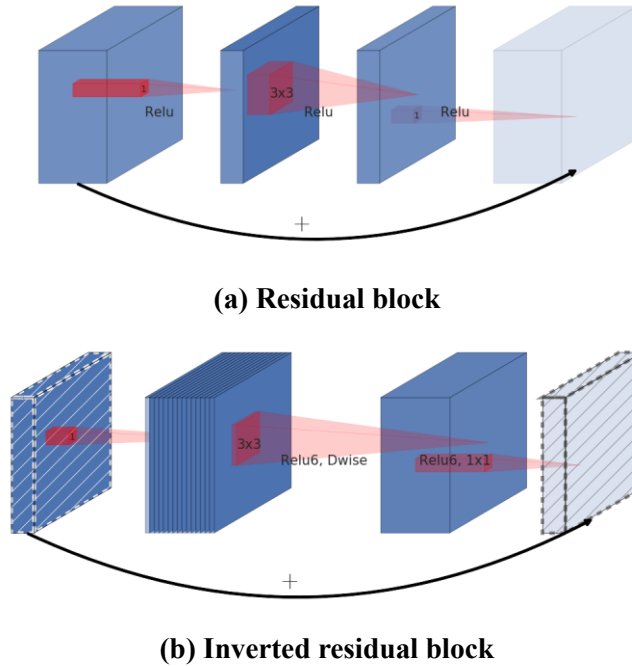


Figure 1. Difference between Residual Block and Inverted Residual

D. Model Architecture

The basic building block for MobileNetV2 is a bottleneck depth-separable convolution with residuals. Figure 2 describes the convolutional blocks for MobileNetV2.

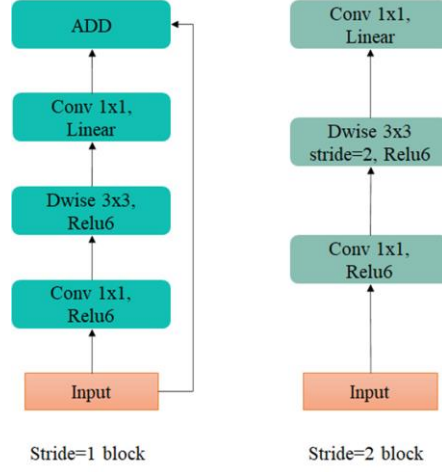


Figure 2. The Convolutional Blocks for MobileNetV2

2.3 YOLOv9

YOLOv9 was released in 2024 as an advancement from YOLOv7 [13]. YOLOv9 introduced a significant advancement in real-time object detection with groundbreaking techniques such as Programmable Gradient Information (PGI) and the Generalized Efficient Layer Aggregation Network (GELAN). This model demonstrates remarkable improvements in efficiency, accuracy, and adaptability, setting new benchmarks on the MS COCO dataset. Due to the integration of PGI, YOLOv9 effectively addresses the issue of data loss in deep networks, thus ensuring the retention of crucial features and the generation of reliable gradients for optimal training results. The use of GELAN yields a network architecture that maximizes parameter utilization and computational efficiency, thus presenting YOLOv9 as an adaptable and high-performing model over a wide range of applications [14]. There are four key concepts that underline the operation of YOLOv9. They include Information Bottleneck, Reversible Functions, Programmable Gradient Information (PGI) and the Generalized Efficient Layer Aggregation Network (GELAN).

A. Information Bottleneck

Information Bottleneck describes the issue of data loss during the feedforward process when data is transmitted through deep networks [15]. As shown in Figure 3 [13], with respect to different architectures, the information provided to the objective function for estimating the loss is lost to varying degrees.

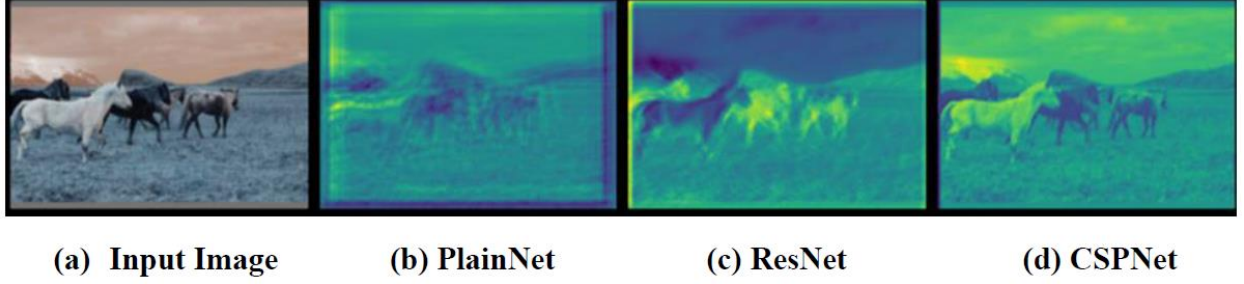


Figure 3. Visualization results of random initial weight output feature maps for different network architectures: (a) Input image, (b) PlainNet, (c) ResNet, (d) CSPNet

Input data X may cause information loss when going through transformation, as shown in Equation 2:

$$I(X, X) \geq I(X, f_{\theta}(X)) \geq I(X, g_{\phi}(f_{\theta}(X))) \quad (2)$$

where I indicates mutual information, f and g denote transformation functions, while θ and ϕ denote parameters of f and g , respectively. In deep neural networks, $f_{\theta}(\cdot)$ and $g_{\phi}(\cdot)$ respectively represent the operations of two consecutive layers in deep neural network. Equation 2 predicts that as the number of network layers becomes deeper, original data is more likely to be lost. However, the parameters of the deep neural network are based on the output of the network as well as the given target, and then update the network after generating new gradients by calculating the loss function. Typically, the output of a deeper neural network is not entirely capable of retaining complete information concerning the prediction target, thus opening the avenue for the network to implement incomplete information during training. This outcome consequently results in unreliable gradients as well as poor convergence. In YOLOv9, reversible functions are utilized to handle the unreliable gradients issue and to carry out relative analysis.

B. Reversible Functions

When a function r has an inverse transformation function v , it is described as the reversible function shown in Equation 3.

$$X = v_{\zeta}(r_{\psi}(X)) \quad (3)$$

wherein ψ and ζ denote parameters of r and v , respectively. Data input X is converted by the reversible function without losing information, as shown in Equation 4.

$$I(X, X) \geq I(X, (r_{\psi}(X))) \geq I(X, v_{\zeta}(r_{\psi}(X))) \quad (4)$$

The reversible functions present in the network transformation function result in more reliable gradients being obtained for updating the model. Although the reversible property could make deep neural networks with more than a thousand layers to converge properly, for complex situations, finding direct simple mapping functions for mapping data to targets becomes highly difficult. Approximation methods are therefore required to determine the inverse transformation of v and r , in order for the transformed features to retain sufficient information using sparse features [13]. However, application of such approach to a lightweight model, could have negative consequences such as the lightweight model becoming under parameterized to a large amount of raw data, making it is easy for too much important information to be lost in the feedforward stage. A burden therefore exists as per how the lightweight model will accurately filter information $I(Y, X)$ (that maps data X to target Y) from $I(X, X)$. A novel deep neural network training method that can generate reliable gradients to update the model, as well as be suitable for shallow and lightweight neural networks, is thus required.

C. Programmable Gradient Information (PGI)

PGI mainly includes three components, namely the *main branch*, the *auxiliary reversible branch*, and *multi-level auxiliary information*. Figure 4 (d) shows that the inference process of PGI uses the main branch only hence does not require any extra inference cost. The auxiliary reversible branch is designed to completely handle or slow down the problems created by deepening of neural networks. Network deepening causes information bottleneck, which limits the generation of reliable gradients by the loss function. Hence, the auxiliary reversible branch ensures the generation of reliable gradients for backward transmission in the main branch. Multi-level auxiliary information, is designed to solve the error accumulation problem resulting from deep supervision, particularly regarding the architecture and lightweight model of multiple prediction branch.

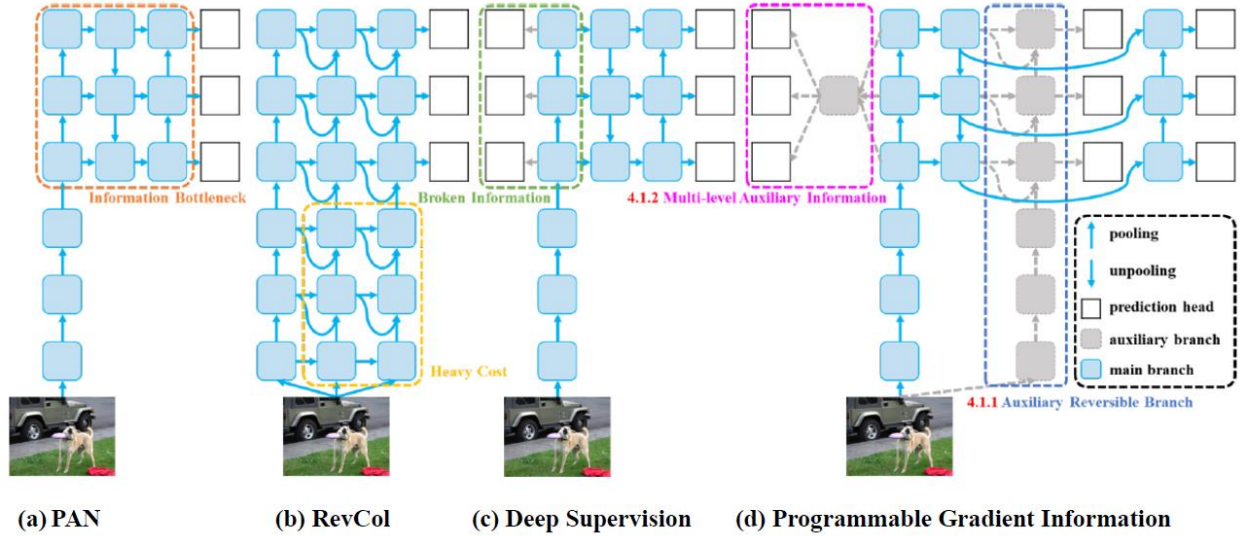


Figure 4. PGI and related network architectures and methods. (a) Path Aggregation Network (PAN), (b) Reversible Columns (RevCol), (c) conventional deep supervision, and (d) Programmable Gradient Information (PGI)

D. Generalized ELAN (GELAN)

Generalized Efficient Layer Aggregation Network (GELAN) is based on the combination of two neural network architectures, namely CSPNet [16] and ELAN [17] designed with gradient path planning. GELAN accounts for factors such as inference speed, lightweight and accuracy. Figure 5 [13] displays the overall architecture of GELAN formed by generalizing the capability of ELAN. Hence, GELAN can use any computational blocks as against ELAN which originally used stacking of convolutional layers only.

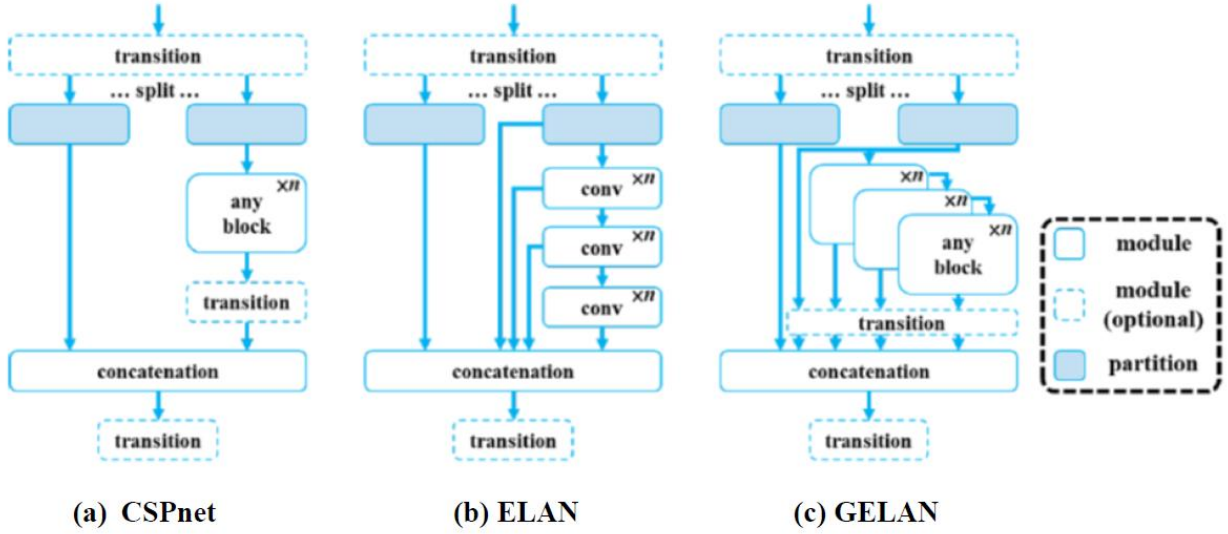


Figure 5. GELAN Architecture

3. MATERIALS AND METHODS

3.1 Importation of Libraries

Various libraries such as os, cv2(for image processing), numpy (for numerical analysis), pandas (for data manipulation), glob (for selecting .xml extension files), xml.etree, Pytesseract (for optical character recognition), torch, torchvision.models, yaml, ultralytics, seaborn, etc. were imported for developing the deep learning models.

3.2 Data Collection

Two datasets were collected for building the models in the license plate detection stage. The Automatic Number Plate Recognition dataset was used to train the MobileNetV2 model, while the Automatic Number Plate Recognition dataset and the Car License Plate Detection dataset were both used to train the YOLOv9c model.

The Automatic Number Plate Recognition dataset has images in JPEG format with bounding box annotations of the car license plates within the images. Annotations are provided in the PASCAL VOC (Visual Object Classes) format. The PASCAL VOC format is used to store annotations for localizer or Object Detection datasets and is implemented by different annotation editors and tools to annotate, modify and train Machine Learning models. For each image, there is an XML annotation file that contains classes, image details, rotation, bounding box details and other data. In order to apply the dataset for training, it was required to present it in an array format. The diagonal points of the bounding box (i.e., $xmin$, $ymin$, $xmax$, $ymax$) respectively were obtained from the labelled images. The bounding box information was available in XML format, hence it was extracted and eventually converted to an array using Pandas. The dataframe consisted of 225 samples and 7 features namely *imagepath*, *xmin*, *xmax*, *ymin*, *ymax*, *width* and *height*. The *glob* library was first used to get all the XML files while *xml.etree* python library was used to parse the data from XML.

The Car License Plate Detection dataset contains 433 images of license plates with bounding box annotations of the car license plates contained within the images. Annotations are also provided in the PASCAL VOC format hence similar data extraction method for the Automatic Number Plate Recognition dataset above was also applied for this dataset.

3.3 Data Processing

In developing the MobileNetV2 model, the dataset was first verified if the bounding box information (i.e., $xmin$, $xmax$, $ymin$, $ymax$, coordinates) for each region of interest (namely the license plates) in the images, as provided in the dataset were correct. An ImageData class was created for the dataset and the data was loaded into the Pytorch environment and converted into a trainable array form of *channel*, *height* and *width*. The images were resized according to height=224, width=224 and normalization was carried out on the images by dividing the $xmin$ and $xmax$ coordinates by the width, then dividing the $ymin$ and $ymax$ coordinates by the height. Thereafter, each image was converted into a tensor in Pytorch and coded as 2 for channel, 0 for height and 1 for width, with all datatypes being float32. A train test split ratio of eighty to twenty was adopted, realizing 180 samples for training the model and 45 samples for validation and the indices were then reset. The ImageData class earlier created was passed into DataLoader, a generator that generates based on batch size. A batch size of 13 was adopted and the shuffle argument was set to *true*, while other default parameters for DataLoader were adopted. Figure 6 displays samples of the images contained in the Automatic Number Plate Recognition dataset.



Figure 6. Image Samples in Automatic Number Plate Recognition Dataset

For the YOLOV9c model build, the two datasets were combined and additional 4 features were created, namely *center_x*, *center_y*, *bb_width* and *bb_height*. Train_test split was carried out with a test size of 0.1 realizing 592 samples for training the model and 66 samples for validation. The root paths for images and labels were defined and the yaml file was realized for YOLO.



Figure 7. Image Samples in Car License Plate Detection Dataset

3.4 Model Building

3.4.1 MobileNetV2

Since MobileNetV2 was pre-trained on ImageNet, the default number of classes are 1000. In our work, we reduced the number of classes to 4 to correspond with the number of classes we are predicting (i.e., $xmin$, $xmax$, $ymin$ and $ymax$). The *BatchNorm2d* component of the model was not trained while other default parameters were set. Early stopping class was created with patience=10 and other parameters set to default. The DataTrainer class was created which takes in the model and DataLoader set, having inputs of optimizer, loss function, learning rate, scheduler and early stopping all initialized. A train_model method (which trains the model and returns the training_loss) as well as a validate_model method (which validates the model and returns the val_loss) were both created under the DataTrainer class. After training, the fit () method was also created with epoch parameter. The loss function was set to RMSE (Root Mean Square Error), Adam optimizer set to learning rate of 0.001, scheduler was set with ReduceLROnPlateau having arguments such as patience=2 and learning rate decrease by factor=0.1, early_stopping was set with arguments patience=10, min_delta=0.01 and restore_best_weights=True. The MobileNetV2 model was set to train for 50 epochs but there was early stopping at epoch 25 and returning of Best Model Weights. The ability of the trained model to detect the coordinates of license plates in an image was thereafter tested using two random images from Google and one test image from the Automatic Number Plate Recognition dataset.

3.4.2 YOLOV9c

The YOLOV9c model was obtained from ultralytics library and the batch size was set to 15, number of epochs set to 30 with other default parameters remaining intact. A data.yaml file with path to train and validation was created and the performance of the YOLO model was tested using the same images provided for the MobileNetV2 model testing.

4. MODEL TESTING

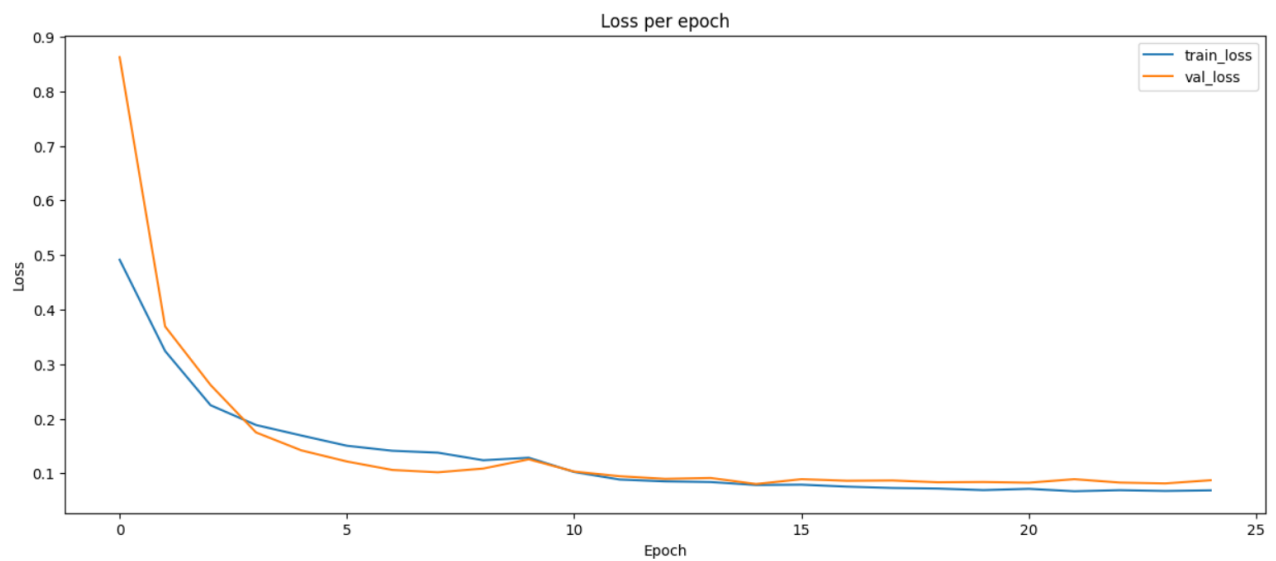
After building the models they were tested using two random images from Google and one test image provided in the Automatic Number Plate Recognition Dataset. For MobileNetV2, the output prediction was of a normalized output (as configured during training) which had to be de-normalized and the bounding box information accompanied the test images. After using the deep learning models to obtain the regions of interest, namely the license plates from the images, Pytesseract was employed for optical character recognition. The textual characters in the regions of interest, namely the license plates were extracted for recognition by the developed system.

5. RESULTS AND DISCUSSION

Results from MobileNetV2 training are described in Table 1 with early stopping at epoch 25 and best model weights returned at that epoch. The learning rate was reduced to 0.0001 at epoch 11, reduced to 0.00001 at epoch 18, reduced to 0.000001 at epoch 21 and reduced to 0.0000001 at epoch 24 to achieve improved results. Figure 8 displays the plot of Loss per Epoch for MobileNetV2 training, while the respective outputs for the three test images are described in Figures 9 to 11 for MobileNetV2.

Table 1: Training and Validation Loss for MobileNetV2

EPOCH	TRAINING LOSS	VALIDATION LOSS
1	0.4914	0.8625
2	0.3241	0.3691
3	0.2248	0.2619
4	0.1885	0.1749
5	0.1693	0.1420
6	0.1505	0.1217
7	0.1413	0.1062
8	0.1378	0.1019
9	0.1240	0.1088
10	0.1286	0.1257
11	0.1024	0.1032
12	0.0886	0.0947
13	0.0853	0.0899
14	0.0840	0.0915
15	0.0786	0.0804
16	0.0792	0.0893
17	0.0756	0.0864
18	0.0732	0.0869
19	0.0721	0.0836
20	0.0693	0.0841
21	0.0716	0.0828
22	0.0671	0.0892
23	0.0691	0.0831
24	0.0677	0.0815
25	0.0688	0.0873

**Figure 8. Training Loss per Epoch for MobileNetV2 Training.**

Test car



Extracted Plate -> IVMGS |

□



Model Output: [292, 520, 404, 449]

Figure 9. Image One Test Result for MobileNetV2

Test car



Extracted Plate -> □



Model Output: [459, 642, 408, 445]

Figure 10. Image Two Test Result for MobileNetV2



Model Output: [374, 574, 477, 534]

Figure 11. Image Three Test Result for MobileNetV2

Figures 9 to 11 also contain the predicted bounding box information for the three test images (i.e. $xmin$, $xmax$, $ymin$ and $ymax$) respectively. It was observed that recognition of characters in the extracted license plates was perfect for image one. However, for image two, Pytesseract failed to recognize the characters in the detected foreground license plate, probably owing to the slant position of the plate. For image three, there was a misplacement of just one character.

Training results for YOLOv9c were presented in terms of mean average precision, precision and recall metrics. The mean average precision (mAP) is the integral over the precision $p(0)$ as shown in Equation 5 [18].

$$mAP = \int_0^1 p(0)d(0) \quad (5)$$

where $p(0)$ is the precision of the license plate detection.

Precision can be calculated by dividing the number of correctly detected license plates by the total number of detected license plates.

Recall can be calculated by dividing the number of correctly detected license plates by the total number of ground truth license plates. Precision and recall in according respect are given mathematically as:

$$Precision = \frac{TP}{TP+FP} \quad (6)$$

$$Recall = \frac{TP}{TP+FN} \quad (7)$$

where TP represents True positives and denotes FP false Positives.

The summary for the YOLOv9c model characteristics is presented in Table 2 while detection results are presented in Table 3.

Table 2: YOLOv9c Model Characteristics

Characteristics	Value
GFLOPs	103.683
parameters	25530003
speed_PyTorch(ms)	13.113

Table 3: Result Metrics for YOLOv9c Model

Metric	Value
mAP50(B)	0.93821
precision(B)	0.94068
recall(B)	0.86364

The predictions for the test images by the YOLOv9c model are displayed in Figures 12 to 14.



Figure 12. Image One Test Result for YOLOv9c



Figure 13. Image Two Test Result for YOLOv9c



Figure 14. Image Three Test Result for YOLOv9c

As shown in Figure 12, the recognition is almost perfect except for the inclusion of just one strange character. The test result for image two though possessing the same limitation of Pytesseract outlined above, presents an interesting development where the YOLOv9c model detects the license plate of a vehicle at the

background which is perfectly recognized by the system, thus overcoming a limitation in [9]. The license plate in test image three is also perfectly detected and recognized by the YOLOv9c model system. These developments propose the YOLOv9c model as a more robust element of the developed deep learning pipeline.

Furthermore, a user purpose application was developed for the detection and recognition system using FastAPI and SQL Alchemy. The performance of the application showed excellent license plate prediction as displayed in Figure 15.

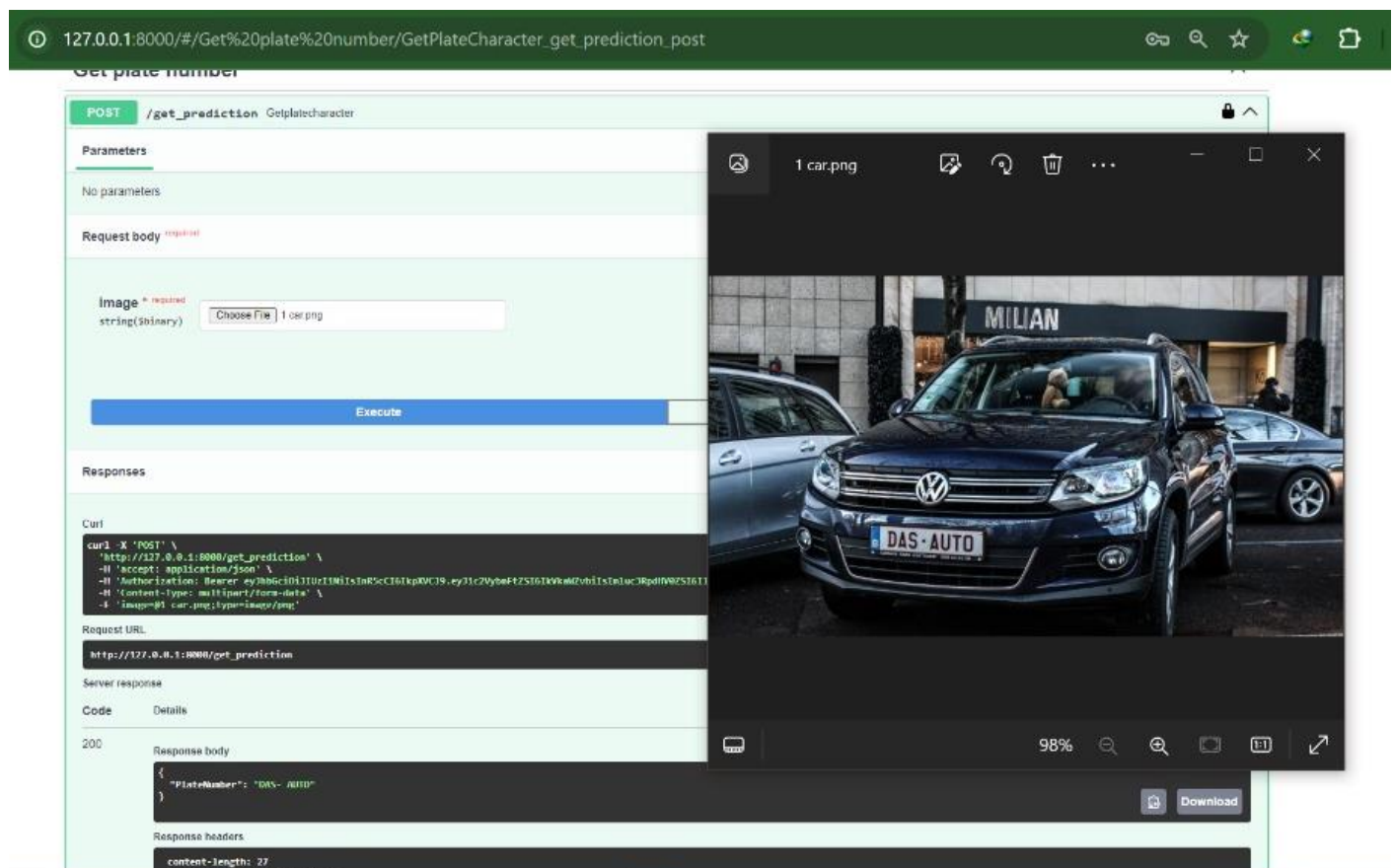


Figure 15. Application for License Plate Deep Learning System

6. CONCLUSION

This work describes the development of a Vehicle License Plate Detection and Recognition System by leveraging Deep Learning methods. Two deep learning models were employed, namely MobileNetV2 and YOLOv9c. Two relevant license plate datasets were used to train the models for license plate detection while Pytesseract was utilized for recognition. Pytorch environment was used to develop the models which displayed satisfactory performance overall. However, test results propose YOLOv9c as a more robust component in the deep learning pipeline. An application for users was also developed using FastAPI and SQL Alchemy. The entire system showed satisfactory performance in detecting license plates and recognizing their character contents.

Acknowledgments

The authors extend their gratitude to Larxel for providing the Car Detection dataset and Aslan Ahmedov for providing the Automatic Number Plate Recognition dataset. Special acknowledgement further extends to Aslan Ahmedov for his suggestions that were of immense inspiration in this work.

REFERENCES

- [1] K.D. Adedayo A.O. and Agunloye ‘Real time Automated Detection and Recognition of Nigerian License Plates via Deep Learning Single Shot Detection and Optical Character Recognition’ Computer and Information Science; Vol. 14, No. 4; (2021) ISSN 1913-8989 E-ISSN 1913-8997.
- [2] P. Batra, I. Hussain, M.A. Ahad, G. Casalino, M.A. Alam, A. Khalique, and S.I. Hassan ‘A Novel Memory and Time-Efficient ALPR System Based on YOLOv5.’ Sensors (2022), 22, 5283. <https://doi.org/10.3390/s22145283>.
- [3] W. Puarungroj and N. Boonsirisumpun ‘Thai License Plate Recognition Based on Deep Learning’ 3rd International Conference on Computer Science and Computational Intelligence (2018).
- [4] Scylla ‘8 Reasons to Use Intelligent License Plate Recognition Systems’ Available at: <https://www.scylla.ai/8-reasons-to-use-intelligent-license-plate-recognition-systems/> (2024) (accessed 6 June 2024).
- [5] R. Laroca, E. Severo, L.A. Zanlorensi, L.S. Oliveira, G.R. Goncalves, W.R. Schwartz, and D.A. Menotti ‘Robust Real-Time Automatic License Plate Recognition Based on the YOLO Detector.’ In Proceedings of the 2018 International Joint Conference on Neural Networks (IJCNN), Rio de Janeiro, Brazil, 8–13 July 2018; pp. 1–10.
- [6] A. Salah ‘Real-time Jordanian license plate recognition using deep learning.’ Journal of King Saud University – Computer and Information Sciences 34 (2022) 2601–2609.
- [7] S.T. Sanjay, D. Yekollu, S. Mandara and G.K. Komal ‘License Plate Recognition for Indian Vehicles based on Image Processing using OpenCV.’ IJIRAE: International Journal of Innovative Research in Advanced Engineering, Volume 10, Issue 03 (2023) pages 112-116 <https://doi.org/10.26562/ijirae.2023.v1003.10>.
- [8] G. Hajare, U. Kharche, P. Mahajan and A. Shinde ‘Automatic Number Plate Recognition System for Indian Number Plates using Machine Learning Techniques’ ITM Web of Conferences 44, 03044 (2022) ICACC-2022. <https://doi.org/10.1051/itmconf/20224403044>.
- [9] Md. T. Rahman, A.N. Merag, A. Muhtasim, Md. W.R. Araf, Md. H.K. Mehedi and A.A. Rasel ‘License plate recognition using machine learning’ International Conference on Images, Signals, and Computing (ICISC 2023). DOI: 10.1117/12.3002521.

- [10] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L. Chen ‘MobileNetV2: Inverted residuals and linear bottlenecks.’ The IEEE Conference on Computer Vision and Pattern Recognition (CVPR); (2018). p. 4510-4520.
- [11] AG. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto and H. Adam ‘Mobilenets: Efficient convolutional neural networks for mobile vision applications.’ CoRR; (2017).
- [12] K. He, X. Zhang, S. Ren and J. Sun ‘Deep residual learning for image recognition.’ CoRR, abs/1512.03385, (2015). 1.
- [13] C-Y. Wang, I-H. Yeh and H-Y-M. Liao ‘YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information’ arXiv:2402.13616v2 [cs.CV] 29 Feb (2024).
- [14] P. Potrimba ‘What is New in YOLOv9? An Architecture Deep Dive. Roboflow Blog’ Available at: <https://blog.roboflow.com/yolov9-deep-dive/> (2024) (accessed 12 June 2024).
- [15] N. Tishby and N. Zaslavsky ‘Deep learning and the information bottleneck principle.’ In IEEE Information Theory Workshop (ITW), pages 1–5, (2015).
- [16] C-Y. Wang, H-Y-M. Liao, Y-H. Wu, P-Y. Chen, J-W. Hsieh and I-H. Yeh ‘CSPNet: A new backbone that can enhance learning capability of CNN.’ In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), pages 390–391, (2020).
- [17] C-Y. Wang, H-Y-M. Liao and I-H. Yeh ‘Designing network design strategies through gradient path analysis.’ Journal of Information Science and Engineering (JISE), 39(4):975–995, (2023).
- [18] Hendry and R.-C. Chen ‘Automatic License Plate Recognition Via Sliding-Window Darknet-Yolo Deep Learning’ Image and Vision Computing, [https://doi.org/10.1016/j.imavis.\(2019\).04.007](https://doi.org/10.1016/j.imavis.(2019).04.007).